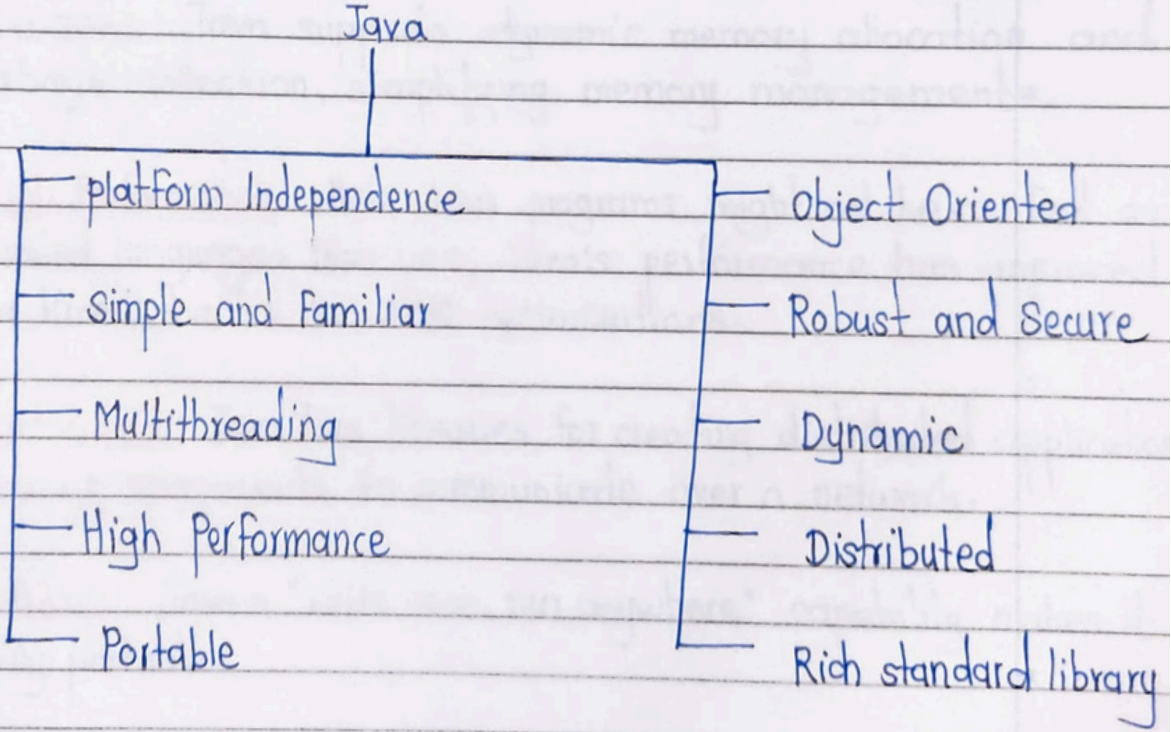
1. Introduction to Java

1.1 What is Java?

Java is a high-level, object-oriented programming language initially developed by Sun-Microsystems (Now owned by Oracle Corporation). It was designed to be platform-independent, meaning that java programs can run on any device or operating system that has a Java Virtual Machine (JVM) installed. This platform independence is achieved. This platform independence is achieved through a "write once, run anywhere" approach, where code written in java can be compiled into bytecode that is then executed on any system with a compatible JVM.

1.2 Features of Java:



1. Platform Independence: Java's bytecode can be executed on any platform with the appropriate JVM.

2. Object Oriented: Java follows the object-oriented programming paradigm, emphasizing encapsulation, inheritance, and polymorphism.

3. Simple and Familiar: Java's syntax is inspired by C++ and C, making it familiar to many programmers.

4. Robust and Secure: Java has feature like memory management, strong type checking, and exception handling to ensure robust and secure programs.

5. Multithreading: Java supports multithreading, allowing multiple tasks to be executed concurrently.

6. Dynamic: Java supports dynamic memory allocation and garbage collection, simplifying memory management.

7. High Performance: While Java programs might not be as fast as compiled languages like C++, Java's performance has improved over time, thanks to JVM optimizations.

8. Distributed: Java has libraries for creating distributed applications allowing components to communicate over a network.

9. Portable: Java's "write once, run anywhere" capability makes it highly portable.

10. Rich Standard library: Java provides a vast standard library

Date :

for various tasks, from data structures to network communication.

1.3 Applications of Java:

1. Web Applications: Java is commonly used for building web applications using frameworks, Javaserver faces (JSF) and servelets.

2. Mobile Applications: Java is used for developing Android applications.

3. Desktop Applications: Java Swing and JavaFX are used to create graphical user interfaces for desktop applications.

4. Embedded Systems: Java's portabillity makes it suitable for embedded systems and internet of Things (IoT) devices.

5. Enterprise Applications: Java EE (Java platform, Enterprise Edition) is used for building - large scale enterprise applicatio

6. Scientific and Research Applications: Java's flexibility and libraries makes it useful for scientific simulations and research projects.

1.4 Java Installation:

To install Java, you need to follow these general steps:

1. Download the appropriate Java. development kit (JDK) for

Date : _____

your operating system from the official Oracle Website or adopt Open JDK

2. Run the installer and follow the installations instructions.

3. Set the 'JAVA_HOME' environment variable to point to the JDK installation directory.

4. Update your system's 'PATH' environment variable to include the 'bin' directory within the JDK installation.

1.5 Java Program :

A simple Java program looks like this :

```
Public class HelloWorld {
    public static void main (String[] args) {
      System.out.println ("Hello, World!");
    }
  }
```

1.6 Internal Details : JVM, JRE, JDK:

1. JVM (Java Virtual Machine) :

JVM is a crucial part of the JAVA platform. It executes JAVA bytecode, providing platform independence, It translates bytecodes into machine code for the host system. It manages memory, handles garbage collection, and supports multithreading. JVM implementations exist

Date :

for various platforms.

2. JRE (Java Runtime Environment):

JRE is the environment required to run Java applications. It includes the JVM, class libraries, and other supporting files. JRE allows users to run Java applications without needing the development tools.

3. JDK (Java Development kit):

JDK is the software package that includes the tools and libraries necessary for developing JAVA applications. It includes the JRE, development tools like the JAVA compiler ('javac'), debugger, and other utilities.

In summary, Java is a versatile programming language known for its platform independence, object oriented nature, and extensive standard library. It finds applications in web, mobile, desktop, enterprise, and scientific domains. The Java ecosystem includes components like JVM, JRE, and JDK that collectively enable the creation and execution of JAVA programs.

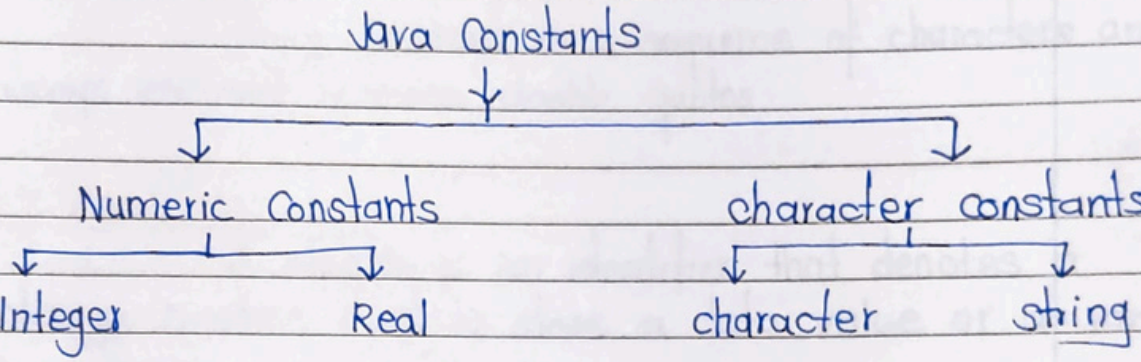
Date :

2. Constants, Variables & Data Types

*

2.1 Constants

In Java constants refer to fixed values that do-not change during the execution of a program. Java supports several types of constants.



1. Integer Constants:

An integer constant means the sequence of digits. There are three types of Integers,

i) Decimal integers consists of digits 0 to 9. Ex: 325, -125

ii) Octal integers consists of any combination of any digits from 0 to 7.
Ex: 027, 123

iii) Hexadecimal Integers - A sequence of digits proceeded by 0x or 0x. They may also include alphabets A to F or a to f.
Ex 0x4, 0x1F, 0xabcd

2. Real Constants:

Integer numbers are insufficient to represent quantities that vary continuously, such as distance, height

Date :

temperatures, prices and so on.

3. character Constant :

A single character constant or character constant contains a single character enclosed within a pair of single quote marks

4. String constant

A string constant is a sequence of characters and always enclosed between double quotes.

2.2 Variables

A variable is an identifier that denotes a storage location used to store a data value or a variable is a basic unit of storage
There are three types of variables:

1) Local variable : Local variable declared inside the body of the method is called local variable.

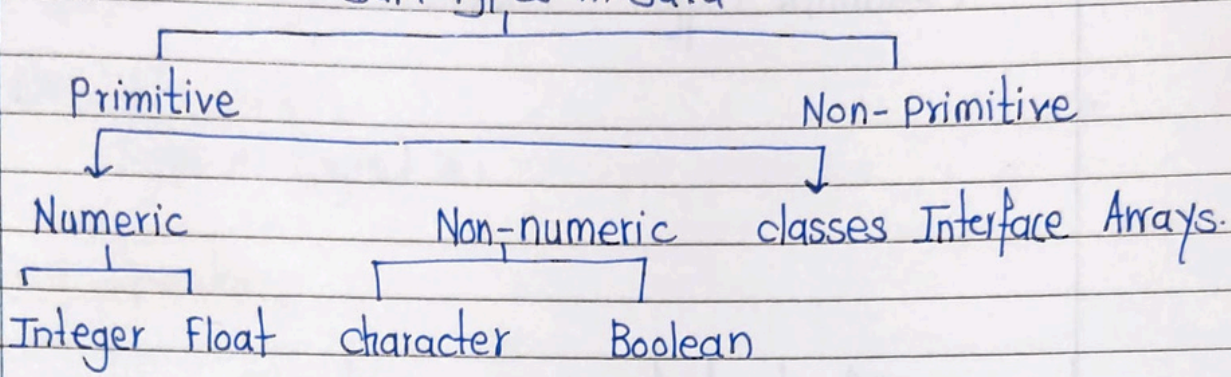
2) Instance variable: A variable declared inside the class but outside the body of the method is called Instance variable.

3) static variable: A variable that is declared as static is called static variable
It cannot be local.
It can create a single copy of static variable and share it among all the instances of the class.

Date :

2.3 Data types

In Java, every variable has a data type. Data type specifies the size and type of data that can be shared.



2.4 Arrays in Java:

Array is a collection of homogeneous or similar data types. elements. Arrays are commonly used to store multiple values in a single variable rather than declaring multiple variables for each value. All arrays are dynamically allocated.

Array declaration

- int [] num;
- string [] cars;
- int [] nums = {1,2,3,4};

Instantiating Array

- Var_name = new type [size];

Date :

Ex: nums = new int [20] ;

2.5 Type Casting

Type casting refers to changing an entity of one data type into another.

syntax: Type variable1 = (type)_Variable2 ;

Ex: int x = 60 ;
byte n = (byte) x ;

2.6 Operators

Operators are classified into following categories:

1. Arithmetic Operators
2. Relational Operators
3. Logical Operators
4. Assignment operators
5. Increment & Decrement
6. Conditional Operators

- **Arithmetic Operators**

1. Addition (+) : Adds two operands togethers
ex: a+b , where a=1, b=2 , a+b =3

2. Substraction (-) : Subtract the second operand from the first operand.
ex: a-b where a=3 , b=1 , a-b = 2

3. Multiplication (*) : Multiplies one operand by another operand
ex: a*b where a=1 , b=2 , a*b = 2

Date : _____

4. Division (/) : Divides the first operand by second operand and returns the quotients.
ex: a/b , where a=20, b=10, a/b = 2.0

5. Modulus (%) : Returns remainder after dividing the first operand by second operand.
ex: a=6, b=2 remainder is 0.

Relational Operators:

When comparison of two quantities is performed depending on their relation certain decisions are made.

1. Equal to (==) : This operator is used to check whether the two given operands are equal or not.

2. Not equal to (!=) : This operator is used to check whether two given operands are equal or not. If not equal, it returns True.

3. Greater Than (>) : This checks whether the first operand is greater than second or not.

4. Less than (<) : This checks whether the first operand is less than second or not.

5. Greater than or equal to (>=) : This checks whether the first operand is greaterthan or equal to second operand or not.

6. Less than or equal to (<=) : This checks whether the first operand is less than or equal to second operand.

Date :

Logical Operators:

1. Logical AND (&&) : Returns true when both conditions under consideration are satisfied.

2. Logical OR (||) : Returns true when one of the two conditions under considerations are true.

3. Logical NOT (!) : Returns true when condition under the consideration is not satisfied.

Assignment Operators :

Assignment Operators are used to assign the value of an expression to a variable (=)

Increment and decrement :

Java has increment (++) which adds one and decrement (--) operator which substracts one. These operators has following forms.

i) Postfix Increment (a++)
ii) prefix Increment (++a)
iii) postfix decrement (x--)
iv) prefix decrement (--x)

Conditional Operators (Ternary operators)

- expr1 ? expr2 : expr3

It works in three steps:

Date :

1) first expt is evaluated for result.
2) If it is true the expression exp2 is evaluated for result.
3) If expt is false, exp3 is evaluated.

2.7 keywords:

Java keywords are also known as reserved words. keywords are special words that have specific meaning and purpose within Java language. They are reserved and cannot be used as variable names or identifiers. keywords play a crucial role in defining the structure and behaviour of programs. keywords are like building block that allow us to create conditional statements loops, functions, classes, handles errors and perform other important operations.

List of Java keywords:

short	do	class	break
continue	default	float	super
boolean	double	public	import
Protected	transient	static	throw
case	native	throws	while
this	return	finally	interface
abstract	new	long	try
if	implements	private	extends
for	int	else	package
void	switch	catch	char
final	byte	instance of	synchronized